

Anchoring Critic-Guided Fine-Tuning of Generative Robot Policies

Anonymous Authors

Abstract—Robot policy refinement should correct failures while retaining behavior that already works. Sparse task rewards make this difficult: a failed attempt does not identify which actions should change, and imperfect critic guidance can disrupt an otherwise useful skill. We present CAST (Constrained Action-Space Transport), a likelihood-free method for learning residual corrections to a frozen generative policy. CAST constructs action targets using critic guidance, soft restoration toward the base, and displacement clipping, then fits the residual by regression. Restoration opposes accumulated departure from pretrained behavior; clipping limits each requested correction. Across eight LIBERO-90 tasks, per-task refinement improves mean success by 9.4 percentage points over the drifting-model base, compared with 0.9 for DICE-RL. On a shared diffusion base for robomimic square, CAST reaches 90% success with approximately 38% fewer environment steps than DICE-RL. A five-task LIBERO ablation shows why the reference matters: removing all restoration yields zero success even with clipping. Initial OpenArm evaluations show gains of 40 and 35 percentage points over the placement and stacking bases. Alternative constraints and transfer studies reveal task-dependent limits. Together, the results support a persistent reference to pretrained behavior as a practical way to preserve competence while allowing adaptation.

I. INTRODUCTION

A robot can perform most of a skill correctly and still fail the task. In our stacking experiments, the robot often reaches, grasps, and aligns the cube but fails to release the gripper. The attempt earns no success reward, although much of the motion is useful. Refinement must therefore learn from the failed outcome while retaining the behavior that brings the robot close to success.

Generative policies learned from demonstrations provide a starting point for this process. Reinforcement learning (RL) can improve them through experience and task feedback [1], [2], but sparse outcomes leave the required correction implicit. A critic predicts future reward, and its action gradients suggest changes to the policy. A low predicted return before a failed release can correctly reflect poor eventual completion without showing that the approach itself was wrong. With limited experience, these gradients may offer unreliable guidance, and optimizing learned values can exploit prediction errors and reduce actual success [3], [4].

The pretrained policy and critic serve complementary roles: the base provides a reference for useful behavior, while the critic proposes how to improve it. We ask *how can refinement retain this reference while allowing changes beyond the base’s competence?* Residual learning offers a natural architecture by adding learned corrections to a frozen policy [5]. Freezing the base preserves its parameters, but the residual can still override its behavior. Limiting individual corrections is also insufficient to limit their accumulated

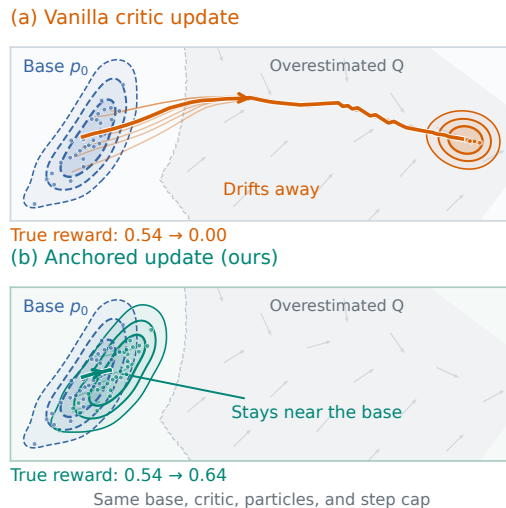


Fig. 1. **Same critic, different policy update.** Both panels start from the same blue base distribution and 384 paired actions. (a) Vanilla critic ascent moves probability mass into the overestimated region (gray). (b) Anchoring permits improvement near the base. Colored contours enclose 50/80/95% of the estimated mass; paths show representative updates in shared, rotated action coordinates. Scores are particle averages. This single-seed toy uses 25 critic-training samples, 60 updates, and a 0.06 step cap. Radius-0.15 per-particle projection provides a fixed-reference diagnostic; CAST uses soft restoration during policy training.

effect: many small changes can eventually move far from the original skill. Fig. 1 illustrates this distinction under a fixed critic.

We introduce CAST (Constrained Action-Space Transport), which pairs a limit on each target correction with a persistent restoring reference to the frozen base. Building on action-target training [6], [7] within DICE-RL’s residual architecture, CAST first asks the critic to propose a change to each sampled action. A soft restoring field opposes departure from the corresponding base action, and clipping caps the combined correction. The residual then learns these explicit targets by regression. Sharing the same noise between current and base actions gives each candidate a consistent reference. The frozen generator needs only to supply samples, so the update requires no policy likelihood. Restoration moderates adaptation while allowing targets to move beyond its soft reference radius.

Our experiments examine whether this design leaves enough freedom to improve. In matched comparisons, CAST gains over 50 percentage points on drifting-base robomimic square. Per-task refinement across eight LIBERO-90 tasks gains 9.4 points over the drifting base, versus 0.9 for the standard DICE-RL actor. The benefit can also be earlier

improvement: on a shared diffusion base for square, CAST reaches 90% success using approximately 38% fewer environment steps, although the two methods eventually reach similar success rates.

Ablations then test the role of the persistent reference. In a five-task LIBERO component study, removing all restoration leads to zero success even when target corrections remain clipped. Restoration variants retain nonzero success, and full CAST improves over every task’s base. These outcomes support a distinction between restricting one requested correction and opposing accumulated departure. The mathematical analysis explains these controls without guaranteeing that the fitted policy remains near the base or improves task success.

Initial OpenArm evaluations connect the study to the motivating failure mode. CAST improves placement and stacking success by 40 and 35 percentage points over the bases. Behavioral cloning (BC) from successful stacking attempts remains competitive with RL, consistent with the difficulty of inferring a useful release correction from delayed outcomes. With 20 trials per method and task, the small differences among refinements do not establish a reliable ranking or a release-specific mechanism.

The broader results identify where the approach is less effective. Alternative constraints lead on individual tasks, and robomimic transport and shared-critic multitask comparisons favor DICE-RL. The DeepMind Control Suite (DMC) also shows that sparse reward alone does not make unanchored refinement fail. A pretrained skill is therefore a useful reference, but its value depends on the task, the critic’s guidance, and how much the policy needs to change.

Our contributions are:

- CAST, a likelihood-free residual update that combines a persistent reference to pretrained behavior with clipped action targets.
- Analyses connecting the update to regularized actor learning and proximity of candidate action distributions, with explicit limits on what target constraints guarantee.
- Evaluations across generative policy classes, simulated manipulation, DMC, and physical robot skills, examining improvements, interaction efficiency, design choices, and failure cases.

II. RELATED WORK

A. Fine-Tuning and Steering Generative Policies

Reward feedback can enter a generative policy through its training objective or its sampling procedure. DPPO optimizes the diffusion denoising process [1]; ReinFlow introduces stochastic flow transitions with tractable likelihoods [2]; and Flow Policy Optimization uses a flow-matching loss surrogate [8]. Diffusion-QL combines diffusion behavior cloning with value maximization [9], while Flow Q-Learning trains a one-step actor with critic guidance and distillation regularization [10].

A frozen policy can also be improved by changing how its samples are obtained or selected. DSRL learns the input noise distribution [11], and V-GPS ranks sampled actions

using an offline-trained value function [12]. IBRL uses imitation-policy proposals for exploration and value bootstrapping [13]. CAST operates on the generator’s output: it learns residual action corrections while retaining the frozen base’s sampling procedure.

B. Critic-Guided Action and Particle Updates

Action-target training separates improvement from policy fitting. DIPO improves stored actions by critic-gradient ascent [6]; Q-Score Matching (QSM) relates the policy score to the critic’s action gradient [7]; and PA-RL combines candidate ranking and local action optimization with supervised fitting [14]. AWR, MPO, and IDQL also use weighted fitting or action selection [15]–[17].

Particle updates connect CAST to Soft Q-Learning’s amortized Stein sampler [18] and drifting models’ displaced-sample regression [19]. Drifting Field Policy combines top- k critic-selected targets with drift losses that anchor behavior [20]. CAST constructs fresh targets for residual regression. Its main variant uses action gradients, restores each candidate toward its same-noise base output, and clips the proposed displacement. We also study value-only fields to examine alternatives to local gradient guidance.

C. Residual Refinement and Behavioral Constraints

Residual RL augments existing controllers with learned corrections [21]. ResFiT extends this approach to frozen action-chunked BC policies under sparse rewards [22]. Policy Decorator bounds residual outputs and progressively introduces exploration [23]; DAWN addresses critic warm-up and scale mismatch [24]. DICE-RL combines residual value maximization, selective behavior regularization, and value-guided action selection [5].

Behavioral constraints also limit exploitation of critic error in offline RL. BEAR restricts actions toward dataset support [25], while TD3+BC adds a cloning penalty to the actor objective [26]. Fisher Decorator constrains residual transport using Fisher geometry estimated from flow-policy scores [27]. CAST instead applies radial restoration and displacement clipping to explicit action targets, requiring only samples from the base. Within DICE-RL, we test whether clipping alone can replace restoration and compare the resulting update with loss-space penalties and hard target projection.

III. PROBLEM STATEMENT

A. Setting

We study reward-based refinement of a pretrained generative policy. An observation s contains the inputs used by the policy, and an action $a \in \mathbb{R}^{Hd}$ is a chunk of H controls with d dimensions each. The frozen generator π_0 maps s and noise z to an action. Following residual RL [5], we learn an additive correction:

$$a_\theta(s, z) = \pi_0(s, z) + r_\theta(s, z), \quad z \sim \mathcal{N}(0, I). \quad (1)$$

An ensemble critic estimates return from replayed experience. Our manipulation tasks use sparse success rewards;

DMC also includes dense rewards. The goal is to improve task performance while retaining useful pretrained behavior. Matched online comparisons share the base, critic-training procedure, and interaction budget. Physical refinement uses fixed experience collected before training.

B. Accumulated Departure from the Base

A critic can assign high values to actions outside its training coverage. Repeatedly following those predictions may move the policy away from actions that succeed [4]. A step cap alone cannot prevent this: $\|a_{t+1} - a_t\|_2 \leq \delta$ permits distance from the initial action to grow with the number of updates. Here t indexes optimization steps. Fig. 1 illustrates the issue by directly updating paired actions under the same critic. In this toy, vanilla ascent moves probability mass into an overestimated region, while a fixed-base projection retains useful behavior nearby.

This distinction motivates two controls: a persistent restoring force that opposes departure from the base, and a cap on each requested target correction. We quantify departure using the residual’s root-mean-square (rms) magnitude, $D_{\text{base}} = \mathbb{E}_{s,z}[\|r_\theta(s,z)\|_2 / \sqrt{Hd}]$, with states sampled from replay. The design question is how to resist unnecessary changes while permitting useful ones. Because a neural actor shares parameters across samples, constraints on its targets do not by themselves guarantee equivalent constraints on its fitted outputs.

IV. METHOD

CAST turns critic guidance into an explicit action target before updating the policy. The critic proposes an improvement, the frozen base provides a persistent reference, and restoration and clipping determine how much of the proposal reaches the target. We retain DICE-RL’s frozen-base residual architecture and off-policy critic-training procedure [5], replacing its actor objective with regression onto these targets. Fig. 2 illustrates the four steps: sample, propose, constrain, and fit.

The equations below describe the main variant, which uses action gradients and a separate base reference for each candidate. All operations use normalized action coordinates with dimension $D = Hd$. We construct each target from the current actor and critic, then hold it fixed while updating the residual. The frozen generator only needs to supply action samples; its likelihood and parameter gradients are not required.

A. Sample Actions and Propose Corrections

Step 1: pair each candidate with its base action. For each replay state s , draw K independent noises z_i . The same noise generates a base action $a_{0,i} = \pi_0(s, z_i)$ and its current residual correction:

$$\bar{r}_i = \text{sg}[r_\theta(s, z_i)], \quad a_i = a_{0,i} + \bar{r}_i, \quad (2)$$

where sg denotes stop-gradient. Each candidate is restored toward its own base action, preserving a reference for different sampled behaviors. For a fixed (s, z_i) , this reference remains unchanged throughout fine-tuning.

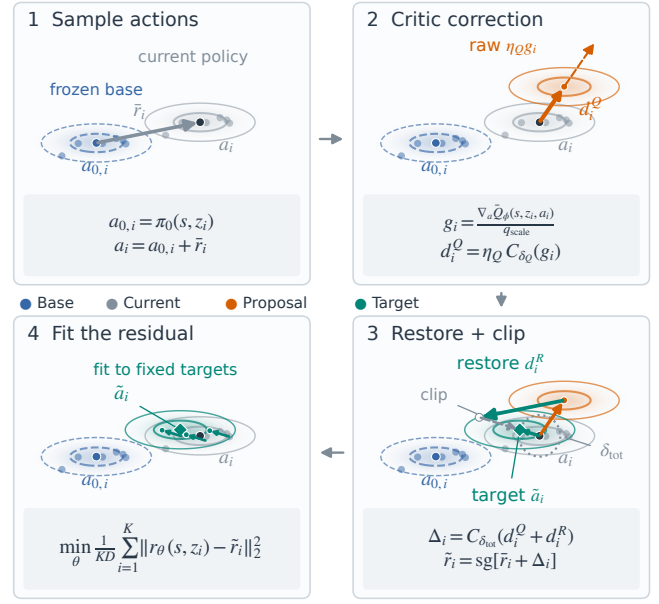


Fig. 2. **CAST constructs a target before updating the actor.** Read clockwise: (1) sample paired base and current actions; (2) propose a clipped critic correction; (3) add restoration and clip the combined displacement; (4) fit the residual to fixed targets. All panels share the same state s , coordinate scale, and base/current distributions. Here $\bar{r}_i = \text{sg}[r_\theta(s, z_i)]$ and the action target is $\tilde{a}_i = a_{0,i} + \bar{r}_i$; sg stops gradients. In step 2, the dashed arrow continues the uncapped request $\eta_Q g_i$ beyond the view. In step 3, restoration is evaluated at the current residual and drawn head-to-tail to show addition. The dashed gray arrow shows total-step clipping; the short green arrow is the accepted correction Δ_i . The dotted circle has radius δ_{tot} around the current action. Step 4 shows the per-state regression objective from (6). This is a constructed schematic with illustrative parameters; green denotes targets rather than a guaranteed post-update policy.

Step 2: ask the critic for a local correction. Let $\bar{Q}_\phi(s, z, a)$ denote the conservative critic-ensemble aggregate; we omit z for noise-independent critics. We divide its action gradient by the detached scale $q_{\text{scale}} = \max(\epsilon, \mathbb{E}_{\text{batch}}[\|\bar{Q}_\phi\|])$, computed over the current candidates, where $\epsilon > 0$ prevents division by zero. This reduces sensitivity to the overall magnitude of predicted values. The proposed critic displacement is

$$d_i^Q = \eta_Q C_{\delta_Q} \left(\frac{\nabla_a \bar{Q}_\phi(s, z_i, a_i)}{q_{\text{scale}}} \right), \quad (3)$$

where $C_\delta(v) = v \min(1, \delta / \|v\|_2)$ and $C_\delta(0) = 0$. The scale η_Q sets the strength of critic guidance. The field cap δ_Q clips the normalized gradient *before* applying η_Q , so $\|d_i^Q\|_2 \leq \eta_Q \delta_Q$. Small gradients retain their magnitude; only those exceeding the cap are rescaled. We differentiate with respect to actions while holding critic parameters fixed. Normalization and clipping control the size of this proposal; its usefulness still depends on the learned critic.

B. Restore Toward the Base and Limit the Step

Step 3: combine improvement with a persistent reference. Clipping the critic proposal limits one step, but repeated steps can still move an action far from its base. We therefore also act on the accumulated residual.

Let $m_i = \|\tilde{r}_i\|_2/\sqrt{D}$ be its rms magnitude. Restoration gives

$$d_i^R = -\eta_0 \tilde{r}_i - \eta_{\text{anc}} \left(1 - \frac{\rho}{\max(m_i, \epsilon)} \right)_+ \tilde{r}_i, \quad (4)$$

with $(x)_+ = \max(x, 0)$. The first term supplies a weak pull toward the base at every nonzero residual. The second adds stronger resistance once departure exceeds the rms radius ρ : it points toward the base with magnitude proportional to the excess distance. Expressing ρ in rms units makes it comparable across action-chunk dimensions.

The radius creates room for adaptation before the stronger pull activates. Inside this dead zone, the weak term remains active. At $\rho = 0$, both terms act on every nonzero residual; as $\rho \rightarrow \infty$, only the weak term remains. Thus the radius controls when stronger restoration begins, while η_{anc} controls its strength.

We evaluate both displacements at the current candidate, add them, and cap the *combined* correction to construct the residual target:

$$\tilde{r}_i = \text{sg} \left[\tilde{r}_i + C_{\delta_{\text{tot}}} (d_i^Q + d_i^R) \right]. \quad (5)$$

Consequently, $\|\tilde{r}_i - \tilde{r}_i\|_2 \leq \delta_{\text{tot}}$. This second cap limits the requested change even when restoration is large. The two controls address different quantities: restoration opposes accumulated departure from the base, while clipping limits the correction requested by one target. The component study in Sec. V-D tests this distinction. Restoration is soft: the combined field and step cap may leave a target outside the base-centered dead zone.

C. Learn from Fixed Targets

Step 4: fit the residual to the corrected actions. The corrected action is $\tilde{a}_i = a_{0,i} + \tilde{r}_i$. Since the base is frozen, fitting this action is equivalent to fitting its residual:

$$\mathcal{L}_{\text{actor}} = \mathbb{E}_s \left[\frac{1}{KD} \sum_{i=1}^K \|r_\theta(s, z_i) - \tilde{r}_i\|_2^2 \right]. \quad (6)$$

Only this mean-squared error updates the actor. The detached targets prevent differentiation through target construction, so the actor learns the requested corrections without taking further derivatives through the critic. Targets are reconstructed for subsequent updates; they do not replace the executed actions stored in replay.

Training and execution. Algorithm 1 shows the online simulation loop. We retain the baseline’s replay mixture, warm-up, exploration, and multi-sample n -step temporal-difference (TD) critic update. Only the critic has a slowly updated target network. In the physical study (Sec. V-C), the same actor construction is applied to saved experience: rollout collection is omitted during training, and corrections are restricted to the nominal executed prefix of each chunk.

At execution, $\pi_0(s, z) + r_\theta(s, z)$ generates candidate actions. Evaluation retains each benchmark’s selection rule, including value-guided selection in the matched LIBERO comparisons. This interface lets the same residual update refine different generator classes using their existing sampling procedures.

Algorithm 1 CAST: online residual fine-tuning

Require: Frozen base π_0 , demonstrations $\mathcal{D}$, candidates per state K , restoration, clipping, and training settings

- 1: Initialize residual r_θ and critic Q_ϕ
- 2: Initialize target critic: $\phi^- \leftarrow \phi$
- 3: Seed replay $\mathcal{R}$ with base-policy rollouts
- 4: **for** each online iteration **do**
- 5: Collect rollouts using the baseline’s warm-up and exploration rules; append transitions to $\mathcal{R}$
- 6: **for** each gradient step **do**
- 7: Sample a minibatch from $\mathcal{R}$ and $\mathcal{D}$
- 8: Form the multi-sample n -step TD loss using Q_ϕ^-
- 9: **if** an actor update is due **then**
- 10: Sample K noises per state; form paired candidates via (2)
- 11: Propose clipped critic displacements d_i^Q via (3)
- 12: Evaluate restoration d_i^R at $\tilde{r}_i$ via (4)
- 13: Cap $d_i^Q + d_i^R$ and detach targets via (5)
- 14: Update θ by target regression via (6)
- 15: **end if**
- 16: Update ϕ with the critic TD loss
- 17: Update ϕ^- by Polyak averaging when scheduled
- 18: **end for**
- 19: **end for**
- 20: **return** Candidate generator $\pi_\theta = \pi_0 + r_\theta$

D. What the Update Controls

How action corrections reach the actor. Target regression provides an interface for processing critic guidance before it changes the network. A local identity shows exactly which correction the actor receives.

Proposition 1: Fix the sampled states and latents, critic evaluation, and normalization scale used to construct targets at θ_{old} . Let $\Delta_i = \tilde{r}_i - \tilde{r}_i$ and let $J_i = \partial r_\theta(s, z_i)/\partial \theta$ be the residual Jacobian at these parameters. Then

$$\nabla_\theta \mathcal{L}_{\text{actor}}|_{\theta_{\text{old}}} = -\frac{2}{KD} \mathbb{E}_s \left[\sum_i J_i^\top \Delta_i \right]. \quad (7)$$

Proof. The targets are detached, so each summand contributes $2J_i^\top (r_\theta - \tilde{r}_i)$. At θ_{old} , $r_\theta - \tilde{r}_i = -\Delta_i$; averaging yields (7). $\square$

The transposed Jacobian $J_i^\top$ maps an action correction into its contribution to the actor gradient. Equation (7) shows that CAST’s clips act on individual corrections *before* that mapping. This differs from clipping the actor’s aggregate parameter gradient.

For $\rho = 0$ or $\rho \geq \epsilon$, the restoring displacement is the negative gradient of the radial potential

$$U(r) = \frac{\eta_0}{2} \|r\|_2^2 + \frac{\eta_{\text{anc}}}{2} (\|r\|_2 - \sqrt{D}\rho)_+^2, \quad (8)$$

with a continuous extension at zero. The two terms penalize total departure weakly and excess departure more strongly. When both clips are inactive, the local actor gradient is $2/D$ times that of the sampled objective with per-action term $-\eta_Q \tilde{Q}_\phi / q_{\text{scale}} + U(r_\theta)$. Thus regression alone does not introduce a new first-order improvement direction. This connection motivates the loss-space hinge comparison; it does not imply equivalence of finite updates with clipping or DICE-RL’s BC filtering.

Why the paired reference has a distributional meaning.

At a fixed state s , let P_θ^s and P_0^s be the candidate action distributions of the residual policy and frozen base, before any value-guided selection. Assume finite second moments. Pairing actions with the same latent z defines a coupling, or joint sampling rule, whose squared distance is $\|r_\theta(s, z)\|_2^2$. It follows that

$$W_2^2(P_\theta^s, P_0^s) \leq \mathbb{E}_z \|r_\theta(s, z)\|_2^2. \quad (9)$$

The squared 2-Wasserstein distance W_2^2 is the minimum expected squared distance over all such pairings. Evaluating one valid pairing gives the upper bound. Residual control therefore connects to proximity of the candidate distributions without requiring a policy likelihood.

Scope of these statements. The target cap bounds a requested correction. Updating a shared network also changes other samples' outputs, so the fitted actor need not remain inside the same ball. Likewise, (9) depends on the *actual* residual and concerns candidates at a fixed state; it does not bound changes after candidate selection or in the states visited by the robot. Proximity also does not establish improvement: a nearby action can follow an incorrect critic direction, and a useful correction may require substantial departure. The experiments test whether these controls preserve success while leaving sufficient room to improve.

E. Alternatives Examined in the Experiments

The comparisons in Sec. V-E examine both how the critic proposes a correction and how departure is constrained. **Value-only guidance** replaces the action gradient with a kernel-weighted attraction–repulsion field over sampled actions. The top- k variant attracts candidates toward the highest-valued current samples and repels them from the current sample cloud. The value-tilted variant uses all candidates as attractors, with weights proportional to $\exp(\hat{q}_i/\tau)$, where $\hat{q}_i$ is the critic value standardized within the state's candidate set and $\tau > 0$ is a temperature; repulsion uses uniformly weighted current samples. The reported value-only recipes also use a weak distributional restoring field toward independent base samples in place of $-\eta_0 \bar{r}_i$. They retain radial restoration and displacement caps, with settings described in the experimental comparisons.

Alternative constraints change where preservation enters the update. The loss-space hinge penalizes $(m - \rho)_+^2$ within DICE-RL's actor objective, using the differentiable residual magnitude $m = \|r_\theta(s, z)\|_2/\sqrt{D}$ and retaining its BC filtering. Hard projection rescales a proposed residual target into $\|\bar{r}_i\|_2 \leq \sqrt{D}\rho$, a ball around the base action. CAST's cap in (5) is centered on the current action. The reported alternatives also differ in auxiliary restoration and clipping settings, so the experiments compare the implemented updates as a whole.

V. EXPERIMENTS

The experiments evaluate three aspects of refinement.

H1 (useful adaptation): CAST can improve task success or interaction efficiency over standard residual refinement.

H2 (persistent reference): limiting target corrections alone is insufficient to retain success in the tested visual manipulation tasks. **H3 (preservation and adaptation):** neither stronger preservation nor value-only guidance consistently improves refinement. We first measure gains, then examine the update choices that support them, and finally test where the manipulation findings do not transfer.

A. Setup and Comparison Logic

Benchmarks and controls. We evaluate state-based robomimic manipulation [28], image-based LIBERO-90 tasks [29], and DMC [30]. In matched simulation studies, DICE-RL and CAST start from the same pretrained checkpoint and share the residual architecture, critic-training procedure, replay recipe, and interaction budget. DICE-RL maximizes critic value with selectively applied BC regularization; CAST denotes the same residual system with our action-target update. The comparison therefore concerns complete actor objectives, including their weighting of replay states. Collected experience and learned critics can diverge as the policies change. External methods and physical refinement use the separate protocols described below.

Outcomes and replication. Task success is reported as a percentage and differences as percentage points (pp). We measure improvement beyond the base, success relative to DICE-RL at a fixed budget, and interactions needed to reach a common success threshold. Robomimic uses 300-episode evaluations; drifting-base results average the last three checkpoints separately for each of three training seeds. LIBERO uses 100 rollouts for each of three evaluation seeds, which measure evaluation variation rather than independent training runs. DMC reports episode return on its original 0–1000 scale, averaged over ten 1000-step episodes. Captions identify incomplete runs and distinguish comparisons with different configurations.

Update settings. The LIBERO recipe uses $K = 16$, $\eta_0 = 0.05$, $\eta_{\text{anc}} = 1$, $\rho = 0.05$ rms, $\delta_Q = 1$, and $\delta_{\text{tot}} = 0.15$. CAST₁ and CAST₂ use critic step scales $\eta_Q = 1$ and 2, respectively, shared across all eight tasks. Each column therefore evaluates one fixed setting across tasks. Robomimic and default DMC use $\eta_Q = 0.5$ and $\rho = 0$; exceptions are identified below.

B. H1: Improving Success and Interaction Efficiency

A useful reference must leave room to improve. Table I compares refinement within each base policy, while learning trajectories distinguish earlier improvement from a higher final success rate.

Refining weak and strong starting policies. On drifting-base square, CAST reaches 91.2–93.0% from a 38.2% base, exceeding DICE-RL on each of three training seeds at 20K updates. Restoration therefore permits substantial changes beyond the base's competence. Can starts at 87.7%, leaving less room to improve, yet CAST reaches 97.8–99.3%. Two can baseline runs end before the common budget, so this row provides a partially matched comparison. Together, these results show improvement from both weak and strong starting policies.

TABLE I

ROBOMIMIC SUCCESS (%). TOP: DICE-RL AND CAST REFINE THE SAME BASE WITHIN EACH ROW. DRIFTING-BASE ENTRIES GIVE RANGES ACROSS THREE TRAINING SEEDS, AVERAGING EACH SEED’S LAST THREE 300-EPIISODE EVALUATIONS; † MARKS TWO CAN BASELINES THAT ENDED BEFORE 20K UPDATES. THE DIFFUSION PAIR IS EVALUATED AT 496K ENVIRONMENT STEPS; z DENOTES VALUE-ONLY CAST. BOTTOM: EXTERNAL SQUARE RESULTS UNDER SEPARATE BUDGETS AND EVALUATION PROTOCOLS. GAINS Δ ARE PERCENTAGE POINTS (PP) OVER EACH METHOD’S OWN BASE; * DENOTES A STRICTER SUCCESS RULE AND $\circ$ AN ONGOING RUN. BOLD MARKS THE HIGHEST TOP-BLOCK SCORE PER ROW AND THE HIGHEST FINE-TUNED SCORE AND GAIN WITHIN EACH EXTERNAL BASELINE GROUP.

Task / base class	Base	DICE-RL	CAST
Can / drifting	87.7	94.0–97.0 [†]	97.8–99.3
Square / drifting	38.2	86.1–90.9	91.2–93.0
Square / diffusion	60.0	96.5	95.9
Square / flow matching	57.0	83.1	93.4^z
<i>External square references (separate protocols)</i>			
Method	Own base	Fine-tuned	Δ (pp)
DIPO [6]*	34.8	15.6	−19.2
QSM [7]*	34.8	0.0	−34.8
DQL*	34.8	0.0	−34.8
IDQL [17]*	34.8	32.7	−2.1
AWR [15]*	34.8	39.3	+4.5
DPPO [1]	60.0	67.8^o	+7.8

The benefit can be earlier improvement. With longer training on drifting square, both actors approach 97–99% near 40K updates. The diffusion pair likewise reaches 95.9% for CAST and 96.5% for DICE-RL at 496K environment steps. Their learning trajectories differ: CAST first reaches 90% at 128/160/160K steps, versus 224/240/256K for DICE-RL, requiring about 38% fewer steps on average. H1 is thus supported by the success attainable within a practical interaction budget, even when final performance converges. This distinction also matters for physical deployment, where additional experience costs time.

A value-only CAST variant reaches 93.4% on flow-matching square, compared with 83.1% for DICE-RL. The interaction budget for this comparison is unspecified, so it supports compatibility with another generator class without establishing a speed advantage. Across base classes, absolute scores also reflect differences in pretraining.

External comparisons provide context. The lower block of Table I reports DIPO, QSM, Diffusion-QL (DQL), IDQL, and AWR implementations adapted by the DPPO authors, using about 24M environment steps. Their stricter chunk-level success rule scores the base at 34.8%, against which gains are computed. QSM and DQL reach 0%, while AWR improves by 4.5 pp. DPPO reaches 67.8% in an ongoing run from a 60.0% base; a separate run with its original architecture reaches 88.7% at 11.7M steps. These outcomes show that more interaction does not ensure useful refinement in the tested configurations. Differences in systems, budgets, and evaluation preclude attributing their ordering to a single CAST component.

Visual tasks reveal both gains and exceptions.

LIBERO tests whether gains extend across visual tasks under shared settings. Each task receives its own residual and critic over a shared multitask base, with task-ID routing at deployment. On the drifting base, CAST₂ raises mean success from 66.1% to 75.5%, a 9.4 pp gain, compared with 0.9 pp for DICE-RL. Both CAST settings improve over the base on all eight tasks. On red mug and ketchup, DICE-RL falls below the bases, whereas CAST₂ improves them by 23.9 and 11.9 pp. The gain thus includes avoiding degradation as well as learning better behavior. DICE-RL still leads on Study-3 caddy.

The flow-matching results show that the benefit depends on the starting policy and update setting. CAST₁ changes mean success from 75.3% to 75.6%; CAST₂ reaches 77.7%, compared with 75.0% for DICE-RL. Strong base skills such as juice placement have little headroom, and none of the refinements improves that task. Limited headroom is not the only issue: on right caddy, the 62.0% base falls to 56.3% under CAST₂. The average gains support H1, while these exceptions show that a reference to the base cannot ensure every proposed correction is useful.

C. Physical Refinement: Improvement and Failure Modes

The physical study evaluates improvement beyond pre-training, with supervised BC as an alternative to reward-based refinement. It also connects the measured success rates to a concrete failure mode: useful intermediate behavior followed by an unsuccessful final release.

Robot and tasks. We use the seven-joint left follower arm and parallel gripper of an OpenArm platform. Chest RealSense D435 and wrist D405 RGB images are resized to 128 × 128 and encoded by a shared ResNet-18; the policy also receives follower joint/gripper positions and a task-language embedding. A flow-matching policy with a diffusion transformer (DiT) backbone uses ten Euler steps to predict 30-step chunks of absolute joint/gripper targets. Deployment issues commands at 15 Hz, requests a chunk every eight periods, limits arm-command changes to 0.6 rad/s, and ends attempts after 20 s.

Both tasks use a hand-made staircase: placing Jigglypuff into the adjacent case and stacking the red cube on the purple cube. Stacking additionally requires releasing the upper cube without disturbing its support. Fig. 3 shows teleoperated demonstrations alongside separate autonomous evaluation results.

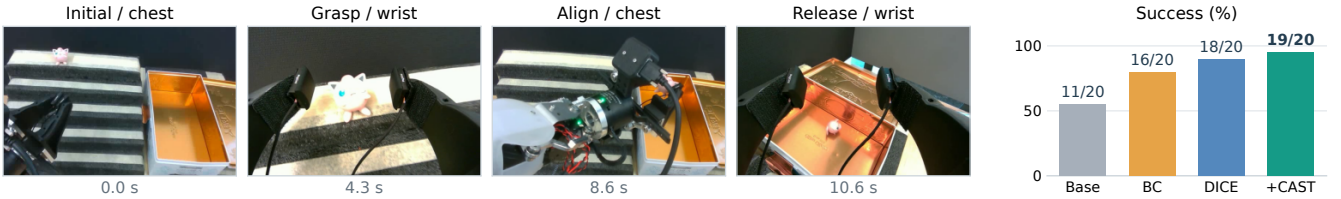
Refinement data and training. Each task uses one shared starting checkpoint. For placement, BC trains on 30 supplementary expert demonstrations; RL uses those demonstrations plus 18 reviewed attempts (11 successes, seven failures). For stacking, BC uses 12 successful attempts; RL uses all 24 reviewed attempts plus 30 original training demonstrations. BC fine-tunes the base policy and encoder, while both RL arms freeze them and train a residual. The BC–RL comparison therefore changes the data, learning objective, and trainable parameters.

TABLE II

SUCCESS (%) ON EIGHT LIBERO-90 TASKS WITH ONE RESIDUAL POLICY AND CRITIC PER TASK, ROUTED BY TASK ID. EACH POLICY IS EVALUATED WITH 100 ROLLOUTS FOR EACH OF THREE EVALUATION SEEDS. EACH BLOCK USES ONE PRETRAINED MULTITASK BASE. CAST₁ AND CAST₂ USE CRITIC STEP SCALES $\eta_Q = 1$ AND 2, RESPECTIVELY, SHARED ACROSS ALL EIGHT TASKS. MEANS AVERAGE EACH FIXED SETTING ACROSS ALL EIGHT TASKS. PARENTHESES GIVE TASK IDS. BOLD MARKS THE HIGHEST SUCCESS PER ROW WITHIN EACH BASE BLOCK.

Task	Drifting base				Flow-matching base			
	Base	DICE-RL	CAST ₁	CAST ₂	Base	DICE-RL	CAST ₁	CAST ₂
Drawer / bowl (8)	81.8	82.0	83.1	85.7	89.7	87.3	90.0	88.7
Stove / pan (21)	69.0	70.3	73.7	75.2	70.0	73.0	69.0	72.7
Drawer / ketchup (32)	61.0	58.7	69.4	72.9	69.0	68.2	73.8	77.3
Basket / juice (53)	78.5	80.0	84.2	85.3	91.7	91.5	89.7	91.3
Plate / red mug (65)	57.3	55.0	82.6	81.2	78.7	80.9	87.8	88.7
Front caddy / book (73)	64.8	66.3	71.0	75.2	68.3	69.0	66.7	78.3
Right caddy / book (75)	57.8	58.3	65.7	64.8	62.0	59.2	60.8	56.3
Study-3 caddy / book (81)	58.8	65.3	60.7	63.5	73.3	70.8	67.0	68.0
Mean	66.1	67.0	73.8	75.5	75.3	75.0	75.6	77.7

(a) Jigglypuff into the case



(b) Red cube onto the purple cube

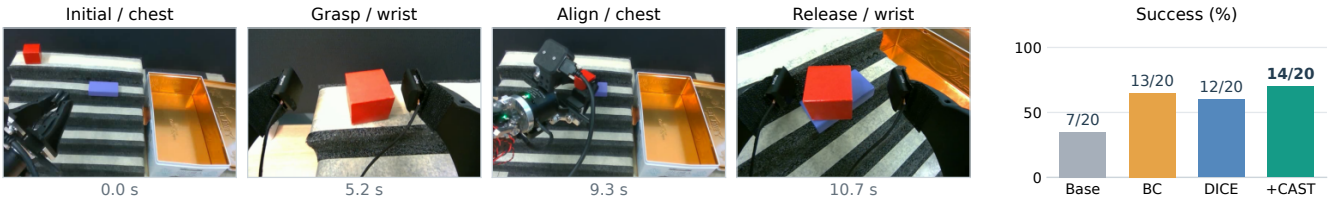


Fig. 3. OpenArm tasks and autonomous success rates (%). The first four panels in each row illustrate a successful teleoperated demonstration: initial state, grasp, alignment, and release. Views alternate between chest and wrist cameras; times are nominal at 15 fps. The final panel reports separate autonomous evaluations, with 20 attempts per method and successful attempts/total above each bar. Base is the pretrained flow-matching policy, BC is supervised refinement, and DICE and +CAST denote DICE-RL and DICE-RL + CAST. The two tasks share the same percentage scale; bold counts identify the highest observed success. Placement poses are randomized on the stairs.

The training protocols specify 40 additional BC epochs and 6,000 updates for each RL arm, with matching DICE-RL/CAST replay indices and critic settings. Both RL arms correct the nominal eight-step executed prefix and use a noise-independent critic because behavior latents were not logged. Training is offline on saved experience. Only successful terminal transitions receive reward 1; other transitions receive 0, and earlier critic targets bootstrap estimates of future return. Image alignment is approximate because frame timestamps were not recorded. The reported autonomous trial counts have not yet been linked to these final training checkpoints.

Observed improvement. Each policy receives 20 autonomous attempts per task. Placement poses are randomized on the stairs. Success requires the toy to remain fully inside the case, or the red cube to remain on the purple cube after release and gripper withdrawal. Paired resets and a fixed

post-release observation interval are not documented.

On placement, success rises from 55% for the base to 80% for BC, 90% for DICE-RL, and 95% for CAST. On stacking, the corresponding rates are 35%, 65%, 60%, and 70%. All refinements show higher success than the bases; CAST gains 40 and 35 pp, respectively. BC remains competitive on stacking, motivating a closer look at the supervision needed to complete the release.

Why stacking can favor BC and CAST. Qualitative observations identify failure to release the gripper as the main stacking failure mode. The robot reaches, grasps, and aligns the cube, but does not open the gripper to complete the stack. The attempt correctly receives a failure outcome, although the preceding motion is useful. Refinement should correct the release while preserving this approach.

A binary terminal reward does not identify which stage caused the failure. The critic estimates expected completion

TABLE III

LIBERO SUCCESS (%) UNDER RESTORATION AND CLIPPING CONFIGURATIONS, WITH THREE TRAINING SEEDS PER CONFIGURATION. *Neither* DISABLES BOTH RESTORING TERMS AND BOTH ACTION-SPACE CAPS; *Clip* ENABLES ONLY THE CAPS. *Restore* ENABLES ONLY DEAD-ZONE RESTORATION ($\eta_0 = 0$), WHEREAS *CAST* USES BOTH RESTORING TERMS AND BOTH CAPS. THE RESTORE-CAST COMPARISON THEREFORE ALSO CHANGES THE WEAK PULL. MEAN AVERAGES THE FIVE TASKS. BOLD MARKS THE HIGHEST SUCCESS IN EACH ROW, INCLUDING THE BASE.

Task	Base	Neither	Clip	Restore	CAST
Red mug	57.3	0.0	0.0	81.0	60.0
Ketchup	61.0	0.0	0.0	38.7	62.7
Bowl	81.8	0.0	0.0	80.7	84.0
Juice	78.5	0.0	0.0	86.0	82.0
Right caddy	57.8	0.0	0.0	42.3	63.0
Mean	67.3	0.0	0.0	65.7	70.3

from limited experience and bootstrapped future values; the actor relies on its action gradients to suggest useful changes. A low value before a failed release can correctly predict unsuccessful completion under the current policy without showing that the preceding action was wrong. Learning an actionable correction from that outcome is a credit-assignment problem: the learner must infer which part of the sequence needs to change.

Success-only BC supplies direct action targets for both the approach and the successful release. This offers a plausible explanation for its 65% success versus DICE-RL’s 60%. DICE-RL can use failed attempts, but its actor follows critic gradients and can relax base regularization when the critic favors a correction. Imperfect local guidance may therefore alter behavior that already works. CAST uses the same critic-training procedure while restoring actions toward the base and limiting target corrections. These controls may reduce disruption of the approach while permitting a release correction; they do not identify release as a special stage or improve the critic’s accuracy directly.

CAST’s 70% stacking success is consistent with this explanation, but exceeds BC by one trial and DICE-RL by two. The release observation therefore motivates a mechanism hypothesis, rather than establishing critic error or a release-specific repair. The observed gains over the bases support further study; larger evaluations with verified checkpoint linkage are needed to compare the refinements reliably.

D. H2: Small Corrections Still Need a Persistent Reference

The component study tests the distinction between a target-step cap and restoration toward a fixed base. Table III compares no controls, clipping only, dead-zone restoration only, and full CAST. Both no-restoration arms set $\eta_0 = \eta_{\text{anc}} = 0$. The restoration-only arm also omits CAST’s weak pull, so its comparison with CAST does not isolate clipping’s marginal contribution.

Both no-restoration arms reach 0% success on all five tasks, even when each target correction is capped. Keeping

TABLE IV

SUCCESS (%) FOR ALTERNATIVE CONSTRAINT AND CRITIC-GUIDANCE DESIGNS. (A) CONSTRAINT IMPLEMENTATIONS USE THREE TRAINING SEEDS, WITH 300-EPIISODE SQUARE EVALUATIONS AND 100 ROLLOUTS FOR EACH OF THREE LIBERO EVALUATION SEEDS. (B) CRITIC-GUIDANCE RECIPES REPORT SQUARE RANGES ACROSS THREE TRAINING SEEDS. MUG AND KETCHUP ARE LIBERO TASKS 65 AND 32. THE BLOCKS RETAIN THEIR OWN CAST REFERENCES; THE IMPLEMENTED ALTERNATIVES ALSO DIFFER IN RESTORATION OR OTHER ACTOR SETTINGS (SEE TEXT). BOLD MARKS THE HIGHEST SUCCESS PER TASK WITHIN EACH BLOCK.

<i>(a) Alternative constraint implementations</i>			
Constraint	Square	Mug	Ketchup
Loss-space hinge	28.0	77.0	65.0
Hard target projection	96.0	93.0	54.0
Full CAST	94.0	78.0	68.0
<i>(b) Critic-guidance variants</i>			
Field	Square	Mug	Ketchup
Gradient	91.2–93.0	60.0	62.7
Top- k	52.5–56.2	57.7	34.1
Value-tilted	56.0–58.4	69.7	37.8

the frozen base in the architecture is therefore insufficient to preserve its useful behavior. Restoration changes the outcome: the restoration-only arm retains nonzero success on every task, and full CAST exceeds each task’s base. This supports H2 in the tested setting: limiting individual target corrections cannot substitute for a persistent reference.

Retaining some success is a weaker goal than improving the skill. Restoration alone raises red mug from 57.3% to 81.0%, but lowers ketchup from 61.0% to 38.7%; its five-task mean is 65.7%, below the 67.3% base mean. Full CAST reaches 70.3% and improves all five tasks over their bases, while restoration alone still leads on red mug and juice. These outcomes establish the practical effect of the tested updates. They do not directly measure critic accuracy or containment of the learned policy’s outputs.

E. H3: Balancing Preservation and Adaptation

Having established the value of a persistent reference, we examine how the constraint and guidance choices affect refinement. Equation 7 motivates a loss-space hinge: without clipping, target regression can express the same first-order direction as a penalized actor objective. The comparison therefore tests the implemented updates, rather than attributing improvement to regression alone.

A harder constraint is not consistently better. Table IV(a) compares CAST with a dead-zone hinge in the DICE-RL objective and projection onto a base-centered target ball. The hinge retains DICE-RL’s BC filtering; the LIBERO projection arm replaces restoration and both displacement caps. These configurations compare complete alternatives, rather than constraint location alone. Projection leads on square and red mug at 96.0% and 93.0%, but drops to 54.0% on ketchup, below its 61.0% base. CAST reaches 68.0% on ketchup and exceeds the corresponding bases on all three tasks.

This ordering is consistent with a preservation–adaptation tradeoff: a hard boundary can protect a useful neighborhood while excluding changes needed for improvement. Soft restoration resists departure while permitting such changes. The experiment does not isolate that geometric explanation, but establishes that a stricter target boundary is not uniformly preferable. The hinge reaches 77.0% and 65.0% on the two LIBERO tasks, yet only 28.0% on square, with training-seed outcomes spanning 3–77%. Multiple constraint designs can support useful refinement; their relative performance depends on the task.

Value rankings and action gradients provide different guidance. Table IV(b) compares gradient guidance with transport toward top- k candidates and transport weighted by standardized critic values. On square, the gradient variant reaches 91.2–93.0%, while both value-only variants remain below 60%. On ketchup, both value-only variants fall below the 61.0% base. Ranking sampled candidates identifies preferred actions; an action gradient proposes a local change to each candidate. These operations need not yield equally useful corrections.

Red mug reverses the ordering: value-tilted transport reaches 69.7%, versus 60.0% for gradient guidance. Thus analytic gradients are not uniformly preferable either. The field recipes also differ in restoration settings, so this comparison does not isolate the source of critic guidance. It shows that both the proposed correction and the controls applied to it matter when evaluating a refinement rule.

The radius determines when stronger restoration begins. Single-training-seed sweeps examine this balance more directly. With $\rho = 0$, the strong restoring term acts on every nonzero residual, yielding 54.9% on mug and 60.1% on ketchup. Positive radii from 0.02 to 0.20 give higher reported success on both tasks, consistent with allowing local corrections before the stronger pull activates. These exploratory sweeps do not establish an optimal radius. As $\rho \rightarrow \infty$, the weak pull remains, so this setting still differs from removing all restoration. Together, the comparisons support H3: stronger preservation alone does not explain better refinement.

F. Scope: When the Manipulation Findings Do Not Transfer

DMC tests whether sparse rewards alone explain the manipulation failures. Its one-step bases are distilled from Soft Actor-Critic (SAC) experience, providing a different task family and source of pretraining data. If sparse reward were sufficient to make unanchored refinement fail, the same failure would occur on these sparse tasks.

Sparse rewards do not by themselves imply collapse. Unconstrained refinement improves sparse balance and ball-in-cup in the reported runs, despite its failures on sparse manipulation. CAST raises sparse balance from 90 to 1000 return and dense swing-up from 429 to 715; on ball-in-cup, the unconstrained interim return of 940 exceeds CAST’s 925. The difficulty illustrated by stacking is more specific than sparsity: a delayed outcome may give little guidance about which part of an otherwise useful skill should change. This

TABLE V
DMC EPISODE RETURN AFTER REFINEMENT OF THE SAME FROZEN BASE WITHIN EACH TASK. RETURNS AVERAGE TEN 1000-STEP EVALUATION EPISODES (MAXIMUM RETURN: 1000). THE TARGET INTERACTION BUDGET IS 400K ENVIRONMENT STEPS; ◦ MARKS AN EARLIER, INCOMPLETE RUN, ν HIGH EVALUATION VARIANCE, AND $\dagger$ A TUNED CRITIC STEP SCALE. SWING-UP AVERAGES TWO TRAINING SEEDS; EACH OTHER ROW REPORTS ONE TRAINING SEED. BOLD MARKS THE HIGHEST REPORTED RETURN PER ROW, INCLUDING TIES AND INCOMPLETE RUNS.

Task	Base	Unconstrained	DICE-RL	CAST
<i>Dense reward</i>				
Cartpole swing-up	429	575	318	715
Cartpole balance	378	815 $^\circ$	844	911
Quadruped run	238	97 $^\circ$	283	312$^{\circ\dagger}$
Finger spin	3	0 $^\circ$	187°	187°
<i>Sparse reward</i>				
Cartpole balance	90	880 $^\circ$	843 $^\nu$	1000
Ball-in-cup catch	86	940°	66 $^\circ$	925

inference also depends on the task and available experience. The DMC comparisons do not separately measure these factors or critic reliability.

CAST improves several DMC tasks with $\rho = 0$, showing that a free interior for the strong restoring term is not necessary everywhere. Dense rewards also do not ensure improvement: the unconstrained quadruped run remains below its base at the reported checkpoint. CAST’s quadruped return of 312 uses a tuned $\eta_Q = 1$ and is below budget, so its comparison with DICE-RL’s 283 is interim rather than a converged, matched-budget result.

When preserving the reference can be costly. On robomimic transport, CAST reaches 59.4% versus 91.2% for DICE-RL at matched 2.24M environment steps. This is consistent with restoration restricting a useful correction, although the experiment does not measure how far the best policy lies from the base. Shared-critic multitask LIBERO also favors DICE-RL: best-of-16 success is 72.2%, versus 69.2% for CAST. That DICE-RL result uses its published recipe, making this a system comparison rather than an isolated actor intervention. These cases delimit the benefit of anchoring: a pretrained skill provides a useful reference, but refinement still needs informative critic guidance and sufficient freedom to change the behavior.

VI. DISCUSSION AND CONCLUSION

A pretrained policy provides a reference for useful behavior, while the critic supplies an improvement signal. CAST makes these roles explicit: the critic proposes action corrections, restoration opposes departure from the frozen base, and clipping limits each target correction used to train the residual. Matched simulation studies show substantial success gains and earlier improvement. The component study shows that clipping alone can fail to preserve competence, while physical placement and stacking provide initial evidence for refinement from recorded practice.

The results also clarify the limits of this approach. Alternative constraints lead on individual tasks, and transport and shared-critic multitask comparisons favor DICE-RL. Sparse reward alone does not determine whether restoration is useful. The target cap bounds a requested correction, whereas the distributional bound depends on the actor's actual residual; neither guarantees successful behavior after fitting. Restoration moderates the influence of critic guidance without establishing its accuracy.

A next step is to adapt restoration to measured local reliability and the amount of change a task requires, alongside larger physical evaluations. The central lesson is that a persistent behavioral reference can support useful refinement, provided it leaves room for the corrections needed to complete the skill.

REFERENCES

- [1] A. Z. Ren, J. Lidard, L. L. Ankile, A. Simeonov, P. Agrawal, A. Majumdar, B. Burchfiel, H. Dai, and M. Simchowitz, "Diffusion policy optimization," in *Proc. ICLR*, 2025, arXiv:2409.00588.
- [2] T. Zhang, C. Yu, S. Su, and Y. Wang, "ReinFlow: Fine-tuning flow matching policy with online reinforcement learning," 2026. [Online]. Available: <https://arxiv.org/abs/2505.22094>
- [3] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. ICML*, 2018.
- [4] S. Fujimoto, D. Meger, and D. Precup, "Off-policy deep reinforcement learning without exploration," in *Proc. ICML*, 2019.
- [5] Z. Sun and S. Song, "From prior to pro: Efficient skill mastery via distribution contractive RL finetuning," 2026. [Online]. Available: <https://arxiv.org/abs/2603.10263>
- [6] L. Yang, Z. Huang, F. Lei, Y. Zhong, Y. Yang, C. Fang, S. Wen, B. Zhou, and Z. Lin, "Policy representation via diffusion probability model for reinforcement learning," 2023. [Online]. Available: <https://arxiv.org/abs/2305.13122>
- [7] M. Psenka, A. Escontrela, P. Abbeel, and Y. Ma, "Learning a diffusion model policy from rewards via Q-score matching," in *Proc. ICML*, 2024, pp. 41 163–41 182.
- [8] D. McAllister, S. Ge, B. Yi, C. M. Kim, E. Weber, H. Choi, H. Feng, and A. Kanazawa, "Flow matching policy gradients," 2025. [Online]. Available: <https://arxiv.org/abs/2507.21053>
- [9] Z. Wang, J. J. Hunt, and M. Zhou, "Diffusion policies as an expressive policy class for offline reinforcement learning," in *Proc. ICLR*, 2023.
- [10] S. Park, Q. Li, and S. Levine, "Flow Q-learning," in *Proc. ICML*, 2025, pp. 48 104–48 127.
- [11] A. Wagenmaker, Y. Zhang, M. Nakamoto, S. Park, W. Yagoub, A. Nagabandi, A. Gupta, and S. Levine, "Steering your diffusion policy with latent space reinforcement learning," in *Proc. CoRL*, 2025, pp. 258–282.
- [12] M. Nakamoto, O. Mees, A. Kumar, and S. Levine, "Steering your generalists: Improving robotic foundation models via value guidance," in *Proc. CoRL*, 2025, pp. 4996–5013.
- [13] H. Hu, S. Mirchandani, and D. Sadigh, "Imitation bootstrapped reinforcement learning," in *Proc. RSS*, 2024.
- [14] M. S. Mark, T. Gao, G. G. Sampaio, M. K. Srirama, A. Sharma, C. Finn, and A. Kumar, "Policy agnostic RL: Offline RL and online RL fine-tuning of any class and backbone," 2024. [Online]. Available: <https://arxiv.org/abs/2412.06685>
- [15] X. B. Peng, A. Kumar, G. Zhang, and S. Levine, "Advantage-weighted regression: Simple and scalable off-policy reinforcement learning," *arXiv preprint arXiv:1910.00177*, 2019.
- [16] A. Abdolmaleki, J. T. Springenberg, Y. Tassa, R. Munos, N. Heess, and M. Riedmiller, "Maximum a posteriori policy optimisation," in *Proc. ICLR*, 2018.
- [17] P. Hansen-Estruch, I. Kostrikov, M. Janner, J. G. Kuba, and S. Levine, "IDQL: Implicit Q-learning as an actor-critic method with diffusion policies," *arXiv preprint arXiv:2304.10573*, 2023.
- [18] T. Haarnoja, H. Tang, P. Abbeel, and S. Levine, "Reinforcement learning with deep energy-based policies," in *Proc. ICML*, 2017.
- [19] M. Deng, H. Li, T. Li, Y. Du, and K. He, "Generative modeling via drifting," 2026. [Online]. Available: <https://arxiv.org/abs/2602.04770>
- [20] J. Koo, M. Park, J. Choi, Y. Min, and M. Sung, "Drifting field policy: A one-step generative policy via wasserstein gradient flow," 2026. [Online]. Available: <https://arxiv.org/abs/2605.07727>
- [21] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. Aparicio Ojea, E. Solowjow, and S. Levine, "Residual reinforcement learning for robot control," in *Proc. IEEE ICRA*, 2019, pp. 6023–6029.
- [22] L. Ankile, Z. Jiang, R. Duan, G. Shi, P. Abbeel, and A. Nagabandi, "Residual off-policy RL for finetuning behavior cloning policies," 2025. [Online]. Available: <https://arxiv.org/abs/2509.19301>
- [23] X. Yuan, T. Mu, S. Tao, Y. Fang, M. Zhang, and H. Su, "Policy decorator: Model-agnostic online refinement for large policy model," in *Proc. ICLR*, 2025.
- [24] G. Ma, L. Li, H. Wang, Z. Liu, P.-L. Bacon, and D. Tao, "What makes value learning efficient in residual reinforcement learning?" 2026. [Online]. Available: <https://arxiv.org/abs/2602.10539>
- [25] A. Kumar, J. Fu, M. Soh, G. Tucker, and S. Levine, "Stabilizing off-policy Q-learning via bootstrapping error reduction," in *Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.
- [26] S. Fujimoto and S. S. Gu, "A minimalist approach to offline reinforcement learning," in *Adv. Neural Inf. Process. Syst.*, vol. 34, 2021.
- [27] X. Cheng, H. Wang, W. Yuan, Z. Wang, Z. Chen, L. Zeng, and Z. Sun, "Fisher decorator: Refining flow policy via a local transport map," 2026. [Online]. Available: <https://arxiv.org/abs/2604.17919>
- [28] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín, "What matters in learning from offline human demonstrations for robot manipulation," in *Proc. CoRL*, 2021.
- [29] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "LIBERO: Benchmarking knowledge transfer for lifelong robot learning," 2023. [Online]. Available: <https://arxiv.org/abs/2306.03310>
- [30] Y. Tassa, Y. Doron, A. Muldal, T. Erez, Y. Li, D. d. L. Casas, D. Budden, A. Abdolmaleki, J. Merel, A. Lefrancq, T. Lillicrap, and M. Riedmiller, "DeepMind control suite," *arXiv preprint arXiv:1801.00690*, 2018.